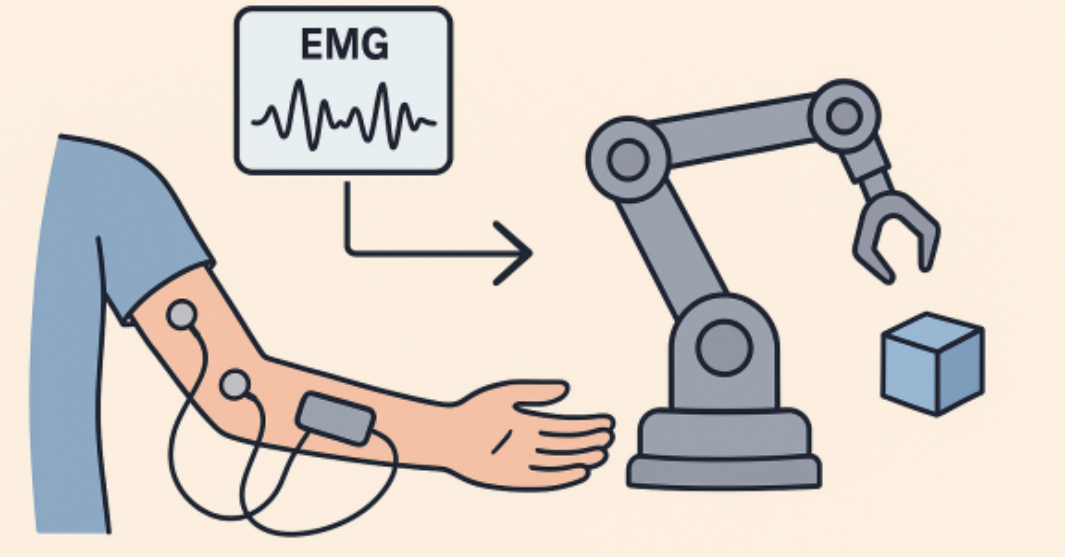
07/04/2026

ENSEA
Ecole Nationale Supérieure de l'Electronique et de ses Applications

Rapport projet semestre 8

Contrôle d'un robot par signaux EMG



Réalisé par Ilyas ATIG, William BEAUSON-MEYER et Laure WOLFF

Deuxième année ENSEA

Responsable: M. Ayoub JEBRI

2025 - 2026

LISTE DES FIGURES

Figure 1 : Software hardware implementation.....

Figure 2 : Acquisition du signal par le capteur Myoware (configuration bipolaire).....

Figure 3 : Numérisation du signal EMG par une entrée ADC de l'ESP32.....

Figure 4 : Simulations initiale et finale du capteur EMG.....

Figure 5 : Routage et dimensions du PCB.....

Figure 6 : Visualisation 3D et PCB réel réalisée.....

Figure 7 : Logique d'Hystérésis et "Télérupteur".....

Figure 8 : Transfert de la commande textuelle via le réseau Wi-Fi.....

Figure 9 : Diagramme de comparaison des architectures (Target Architecture atteinte).....

Figure 10 : Articulations du robot.....

Table de matière

1. Introduction **4**

1-1. Récapitulatif de l’objectif du projet 4

1-2. Analyse à la fin du semestre 7 4

1.3 - Objectifs fixés du semestre 8 5

2. Avancée du projet **6**

2-1. Architecture final du projet 6

2-1-1. Diagramme fonctionnement global 6

2-1-2. Flux d'Information (Information Flow) 8

2-1-3. Choix Technologiques 9

2-2. Communication Réseau 11

2-3. Evolution du capteur EMG 11²

2-3-1. Simulation LTSpice 11

2-3-2. Conception Kicad 13

2-3-3. Réalisation et soudure du PCB 14

2-4. Architecture de Contrôle et Traitement Logiciel 15

2-4-1. Traitement embarqué (Edge Computing) sur l'ESP32 15

2-4-2. Communication TCP et Serveur ROS 16

2-4-3. Modes de contrôles 18

2-4-4. Sécurité et gestion des erreurs 19

2-5. Résultats du projet à la fin du semestre 8 21

3. Conclusion **22**

3-1. Bilan des problématiques rencontrées 22

3-2. Perspectives 22

4. Annexes **24**

1. Introduction

1-1. Récapitulatif de l’objectif du projet

Ce projet vise à développer un système de contrôle robotique basé sur des signaux électromyographiques (EMG). L'objectif est de capter l'activité musculaire de l'utilisateur à l'aide de capteurs EMG, puis de traiter et classifier ces signaux afin de générer des commandes appropriées pour un bras robotisé.

La première étape a consisté à concevoir un capteur EMG capable d'acquérir et de filtrer les signaux bioélectriques pour en extraire des caractéristiques fiables. Ensuite, un algorithme de traitement (détection de niveaux de contraction) a été mis en œuvre pour piloter le robot en fonction du muscle sollicité.

1-2. Analyse à la fin du semestre 7

Le but du semestre précédent a été de s’initier aux différents concepts de ce projet, de les comprendre et d'apprendre à les manipuler.

Pour cela, d’importantes recherches ont été menées pour mieux appréhender l’origine d’un signal EMG et son amplitude pour un muscle du bras humain (5 µV - 1 mV), ainsi que pour la prise en main du robot Niryo et de l’environnement ROS.

À la fin du semestre 7, l’ensemble des choix techniques ont été validés et un premier prototype de capteur EMG (non fonctionnel) a été réalisé. La prise en main du robot et les prémices de la communication ont été réalisées avec succès.

1.3 - Objectifs fixés du semestre 8

Ce semestre avait pour but de finaliser le projet en réalisant plusieurs objectifs :

1. **Réaliser un capteur EMG fonctionnel** afin de pouvoir étudier la contraction de plusieurs muscles pour réaliser des mouvements plus réalistes avec le robot.

2. **Lier le robot au microcontrôleur ESP32** afin de transmettre les mouvements et d'être indépendant de l'ordinateur portable (établir une connexion TCP entre le robot et le microcontrôleur).

3. **Coder des algorithmes** afin de rendre possible la reproduction des mouvement du bras par le robot

2. Avancée du projet

2-1. Architecture final du projet

2-1-1. Diagramme fonctionnement global

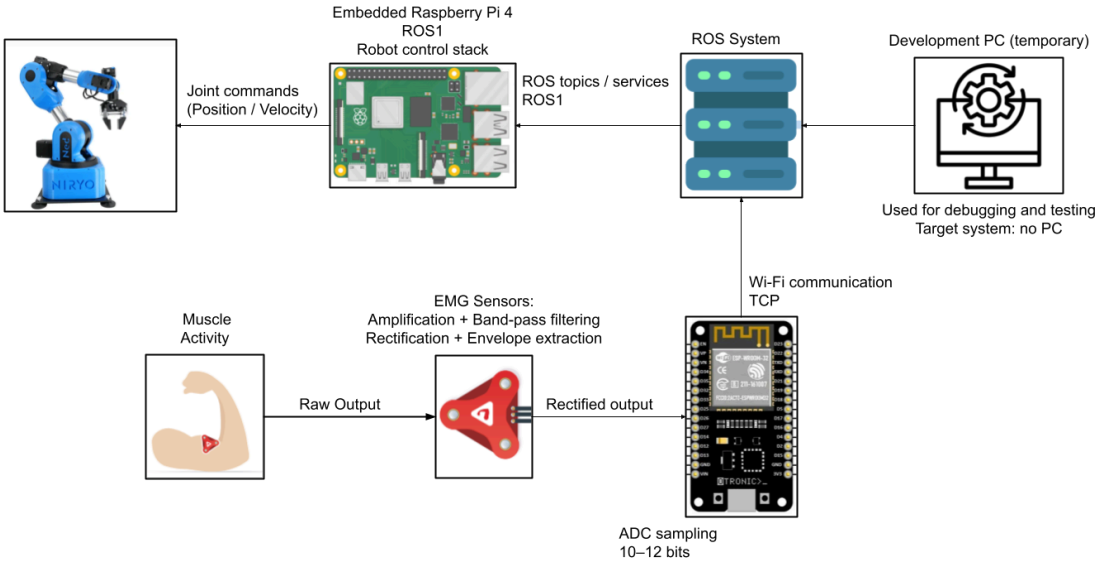


Figure 1: Software hardware implementation

Le schéma fonctionnel global, basé sur la méthode recommandée, relie les composants clés comme suit :

- **Acquisition du signal :**

  Le muscle (agoniste/antagoniste) produit une activité électrique.

- **Capteur EMG :**

  Les électrodes captent cette activité, et le capteur MyoWare, ou un capteur similaire (avec la configuration bipolaire recommandée), convertit le signal brut en un signal analogique préconditionné (en « enveloppe » dans l'exemple MyoWare).

➢ **Microcontrôleur (ESP32) - *Acquisition et Traitement embarqué* :**

Il lit le signal analogique brut provenant du capteur EMG via son convertisseur analogique-numérique (ADC en résolution 12 bits, valeurs de 0 à 4095).

Évolution de ce semestre : L'ESP32 ne se contente plus de transférer un flux de données brutes. Il intègre désormais une logique de traitement du signal local. Un algorithme de détection par seuils (Hystérésis) couplé à une machine à états (logique "télérupteur") analyse l'intention musculaire en temps réel et génère des commandes textuelles (ex: "FLEX", "RELAX", "OPEN", "CLOSE").

➢ **Connexion (Wi-Fi et Protocole TCP/IP) :**

Assure la transmission de données sans fil et sans perte entre le microcontrôleur et le robot. L'ESP32 se connecte directement au point d'accès Wi-Fi (Hotspot) du robot et agit comme un client TCP. Il transmet instantanément les commandes de décision vers un port d'écoute spécifique (Port 5005) du système hôte.

➢ **Système ROS (Raspberry Pi du robot Niryo Ned) - *Serveur de réception* :**

Plutôt que de transiter par une architecture complexe de publication sur des *Topics* (comme /emg/envelope précédemment), nous avons optimisé la latence en déployant un nœud ROS sous forme de Serveur TCP direct (écrit en Python). Ce serveur écoute en permanence les requêtes entrantes provenant de l'ESP32.

➢ **Nœud de Contrôle Robotique :**

Ce module est fusionné avec le serveur de réception. Dès la réception d'une chaîne de caractères explicite (ex: "CLOSE"), le script utilise directement l'API officielle du robot (niryo_robot_python_ros_wrapper). Il convertit cet ordre en instructions matérielles immédiates, qu'il s'agisse de piloter l'outil de préhension (open_gripper / close_gripper) ou d'exécuter un mouvement articulaire pré-enregistré dans l'espace articulaire via move_joints (sans

nécessiter de calcul de cinématique inverse complexe en temps réel pour cette phase de prototypage).

2-1-2. Flux d'Information (Information Flow)

Dans notre système, le signal EMG traverse plusieurs étapes de traitement, allant du signal physiologique à la commande mécanique :

1. **Génération et Acquisition analogique :** Le système nerveux active les unités motrices. Les électrodes bipolaires mesurent la différence de potentiel sur le muscle. Le capteur EMG amplifie, filtre et redresse le signal brut pour en extraire l'enveloppe (représentative de l'intensité de la contraction).

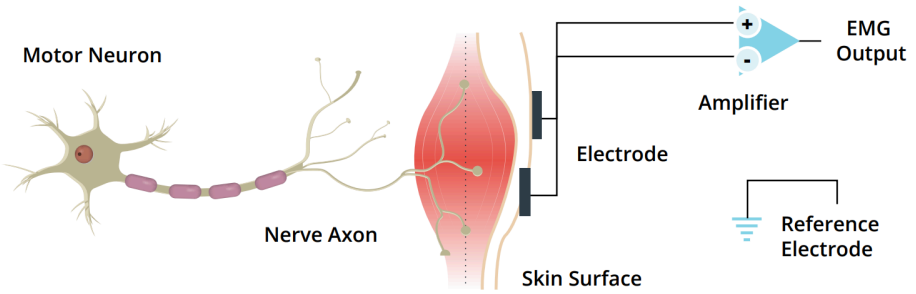


Figure 2 : Acquisition du signal par le capteur Myoware (configuration bipolaire)

2. **Numérisation :** Le convertisseur analogique-numérique (ADC) du microcontrôleur lit la tension analogique et la convertit en une valeur numérique (résolution de 12 bits, de 0 à 4095).

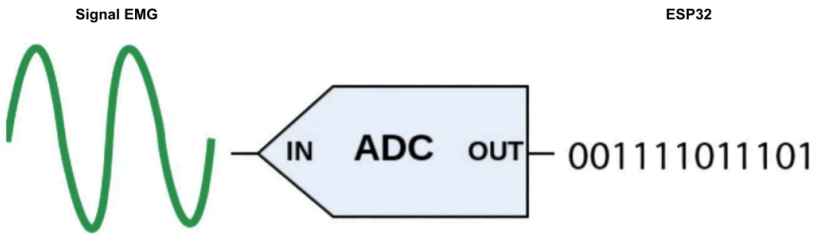


Figure 3 : Numérisation du signal EMG par une entrée ADC de l'ESP32

3. **Traitement embarqué (*Edge Computing*) :** C'est la nouveauté de cette architecture. L'ESP32 n'envoie plus les données en continu. Il analyse le signal localement via un algorithme d'Hystérésis (comparaison avec des seuils Haut et Bas) pour éliminer le bruit. Il applique ensuite une logique d'état ("télérupteur") pour déduire l'intention de l'utilisateur.

4. **Transfert de la décision :** L'ESP32 transmet uniquement une commande textuelle décisionnelle (ex: `"FLEX"`, `"RELAX"`, `"CLOSE"`) via le réseau Wi-Fi, allégeant drastiquement la bande passante.

5. **Exécution robotique :** Le serveur Python du système ROS (Niryo Ned1) reçoit cette commande et exécute les instructions matérielles associées (`move_joints`, `open_gripper`).

2-1-3. Choix Technologiques

Microcontrôleur (ESP32)

L'ESP32 a été conservé car il répond parfaitement au double besoin du projet: une puissance de calcul suffisante pour réaliser le traitement du signal à la source (*Edge Computing*), et une connectivité Wi-Fi native indispensable pour rendre le système sans fil et portable.

Méthode de contrôle (Logique d'états vs Signal Continu)

Contrairement aux hypothèses du semestre précédent qui visaient à envoyer un signal analogique complet et continu (pour un contrôle proportionnel), nous avons opté pour une approche par seuils.

➔ *Justification*

Les tests pratiques ont révélé que maintenir une contraction musculaire constante est inconfortable et génère des signaux bruités dus à la fatigue. L'approche algorithmique de type "télérupteur" (un pic déclenche l'action, le repos réarme le système, un second pic inverse l'action) offre un contrôle beaucoup plus stable et intuitif pour l'utilisateur, tout en s'affranchissant du bruit matériel lié aux électrodes.

2-2. Communication Réseau

La stratégie de communication doit assurer la liaison entre le microcontrôleur et le robot, ainsi que l'intégration du code du projet au système interne du robot lui permettant de se mouvoir. L'objectif était de pouvoir se défaire de la dépendance à un ordinateur et de pouvoir compter seulement sur le capteur, le microcontrôleur et le robot. Nous avons alors opté pour une communication Wi-Fi que nous abordons dans la partie *2-4-2. Communication TCP et Serveur ROS*.

2-3. Evolution du capteur EMG

A la fin du semestre 7, un premier prototype du capteur EMG à été réalisé. Cependant, il n'était pas fonctionnel et donc ce semestre a été consacré à repenser totalement le capteur. Pour cela, une **simulation**, une conception *Kicad* et un **PCB** ont été réalisées.

Le but du capteur EMG est de pouvoir restituer l'enveloppe de la contraction musculaire tout en respectant une tension de sortie comprise entre **0** et **3.3V** afin de ne pas abîmer l'ESP32.

2-3-1. Simulation LTSpice

Avant la réalisation matérielle, une phase de simulation sous *LTSpice* a été indispensable pour valider l'architecture du circuit.

La simulation a permis de décomposer le traitement en trois blocs :

A. **L'amplification d'instrumentation** : Pour extraire le signal différentiel très faible issu des électrodes (de l'ordre de grandeur de 10µV à 1 mV).
B. **Le filtrage actif** : Un filtre passe-bas pour éliminer les bruits et les composantes hautes fréquences inutiles.

C. **Le redressement et lissage** : Utilisation d'un redresseur grâce à un pont de diode suivi d'un filtre passe-bas pour obtenir l'enveloppe du signal. Cette étape a permis d'ajuster les gains et les constantes de temps pour que la sortie ne dépasse jamais le seuil critique des 3,3V, même lors de contractions maximales.

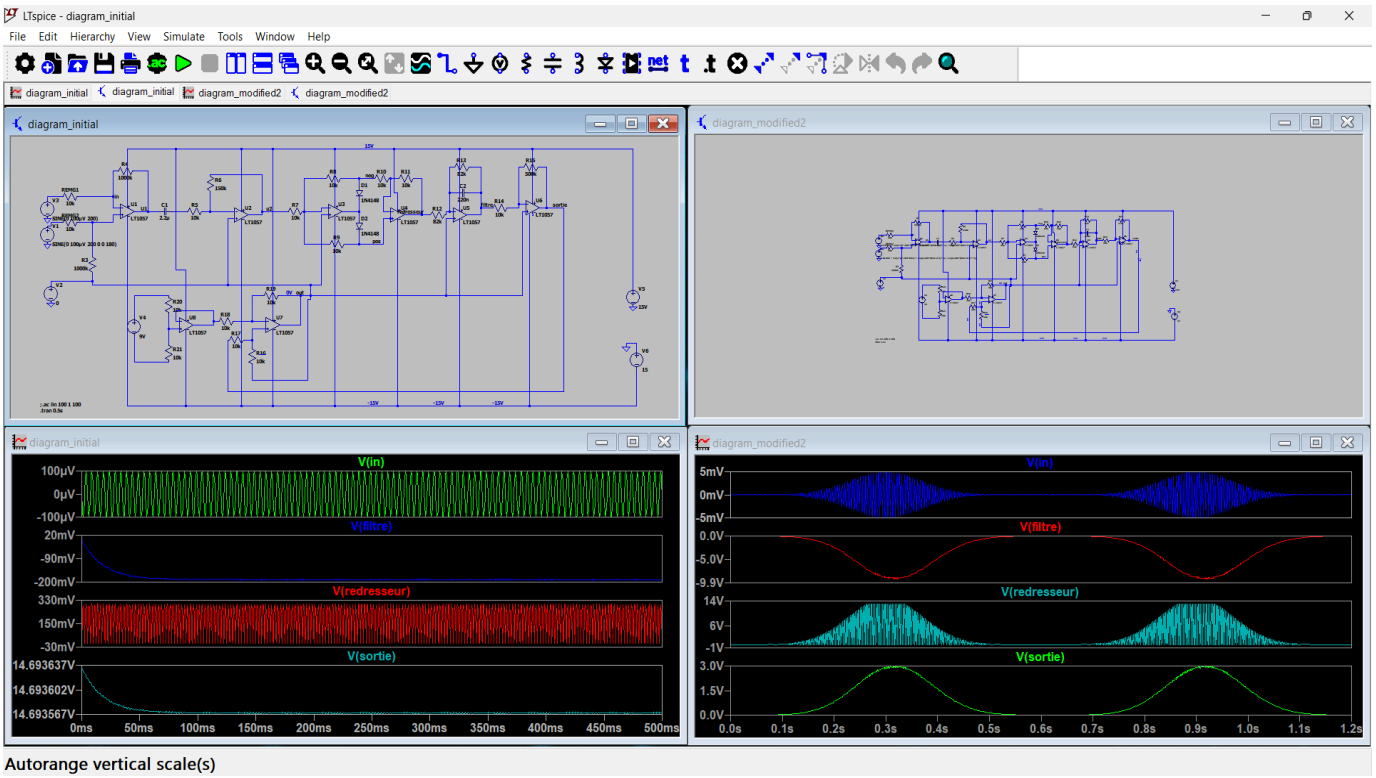


Figure 4 : Simulations initiale et finale du capteur EMG

La phase de simulation a débuté par la modélisation du circuit conçu lors du semestre précédent. Cette étape a été cruciale car elle a permis de confirmer les dysfonctionnements observés physiquement, notamment un signal de sortie saturé et une fréquence de coupure du passe-bas trop faible.

En manipulant les différents blocs du circuit sous *LTSpice*, les corrections suivantes ont été apportées:

1. **Filtrage :** Ajustement des fréquences de coupure des filtres passe-haut (pour supprimer les dérives de la ligne de base) et passe-bas (pour éliminer les bruits hautes fréquences).

2. **Redressement :** Optimisation du redresseur pour assurer une détection d'enveloppe fidèle, transformant le signal alternatif brut en une tension continue représentative de l'intensité de la contraction. Un travail complet à été réalisé sur le redresseur sans seuil afin d'avoir en sortie un signal relativement lisse (même si un traitement au niveau ESP32 sera nécessaire) et exploitable.

La simulation finale a validé le nouveau schéma, démontrant une sortie **stable** et une modélisation de la contraction musculaire. Ce résultat a donné le feu vert pour passer à la conception du PCB sous *KiCad* avec la certitude que l'architecture électronique était théoriquement valide.

2-3-2. Conception Kicad

Après la partie théorique, la conception d'un PCB à été envisagée afin d'avoir la capacité d'étudier plusieurs muscles lors d'un mouvement. Pour cela, en plus du capteur MyoWare, il fallait développer plusieurs capteurs (dans l'idéal 2 autres capteurs).

Pour développer ce PCB, il faut prendre en compte que chaque capteur doit avoir une alimentation de -15V a +15V afin d'alimenter les amplificateurs et d'une batterie de 9V. De plus, plus le capteur est compact plus la place de l'ensemble { microcontrôleur + capteurs + alimentation } peut être optimisée. Dans le cadre où on souhaite rendre autonome le processus (sans ordinateur) le choix d'un PCB compact et optimisé à été choisi. Ainsi le PCB modélisé est un rectangle 3cm x 4cm.

De plus cette taille permet de réduire au maximum les tailles des pistes et ainsi éviter l'ajout de bruit et de perturbations qui viendrait parasiter un signal très faible et ainsi fausser toute l'information.

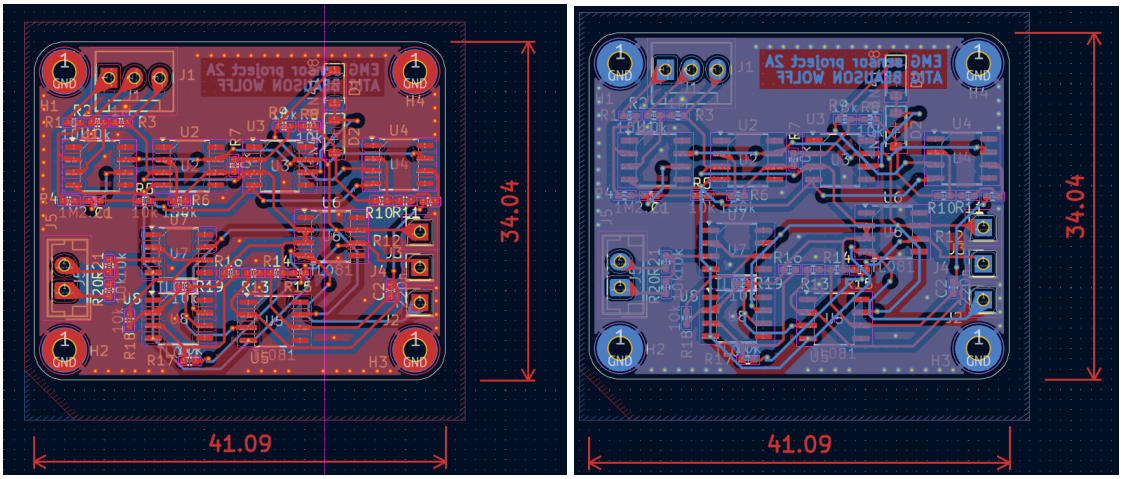


Figure 5 : Routage et dimensions du PCB

Une fois la phase de conception virtuelle finalisée et validée sous *KiCad*, nous avons procédé à la réalisation du capteur en passant à l'étape de fabrication et d'assemblage du circuit imprimé.

2-3-3. Réalisation et soudure du PCB

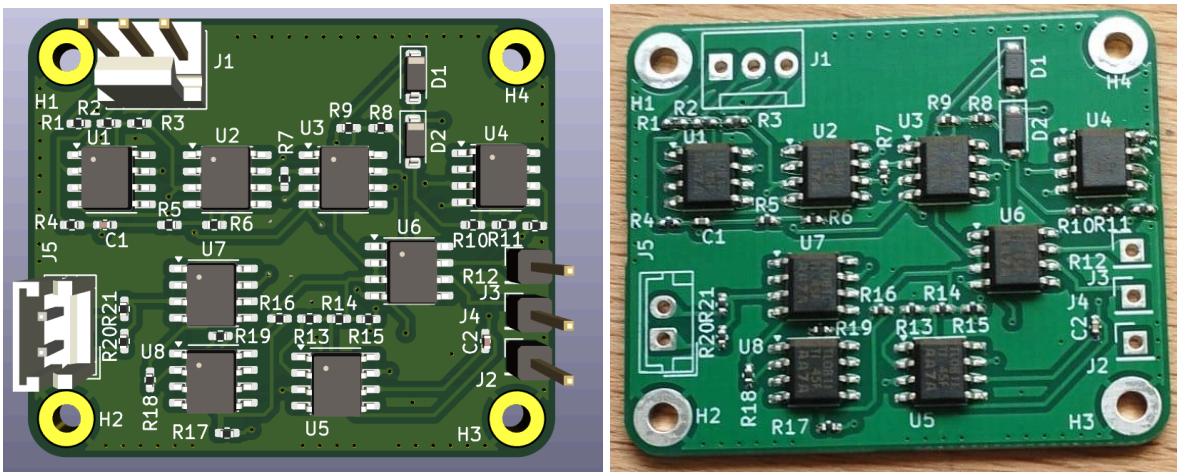


Figure 6 : Visualisation 3D et PCB réel réalisé

Pour souder le PCB, l'utilisation de pâte à braser et du four à refusion a été nécessaire. Malheureusement, pour une question de temps, il n'a pas été possible de le tester malheureusement. Cependant, grâce à la validation préalable par simulation et au soin apporté au routage pour limiter les parasites ainsi qu'une inspection minutieuse à la sortie du four des possibles court-circuits et de la continuité entre

les éléments, ce module constitue une base solide et fonctionnelle pour une potentielle reprise du projet par un autre groupe.

2-4. Architecture de Contrôle et Traitement Logiciel

L'évolution majeure de ce semestre réside dans la refonte totale de la chaîne de communication. Pour pallier les problèmes de latence et de mouvements saccadés identifiés au semestre précédent avec la bibliothèque PyNiryo classique, nous avons opté pour une architecture distribuée s'appuyant sur l'*Edge Computing* et le protocole TCP/IP.

2-4-1. Traitement embarqué (Edge Computing) sur l'ESP32

Notre objectif initial était d'établir une relation **proportionnelle** où l'amplitude du muscle se transforme en consigne de vitesse. Cependant, après test de cette méthode qui s'est avérée instable, nous l'avons abandonné au profit d'une autre.

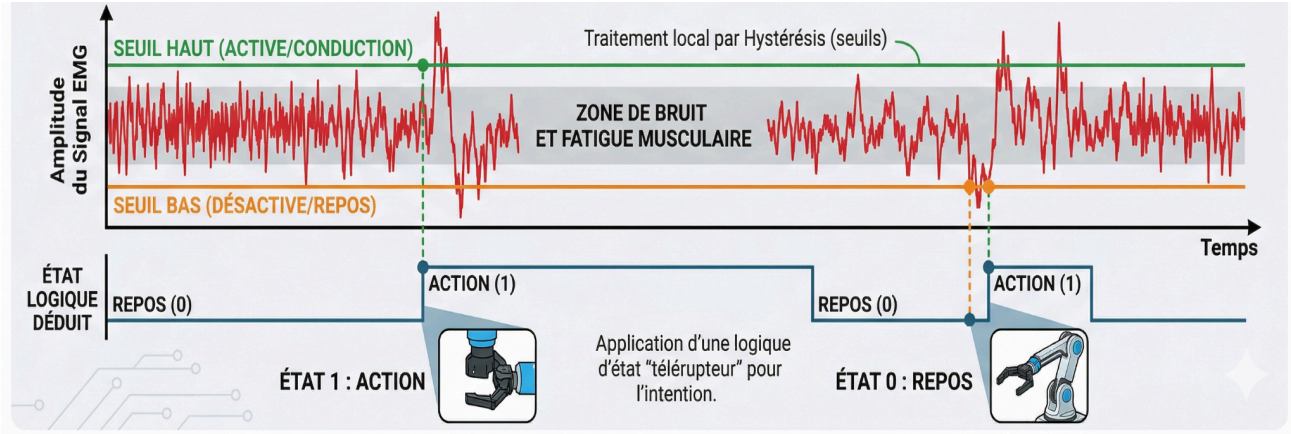


Figure 7 : Logique d'Hystérésis et "Télérupteur"

Plutôt que de saturer le réseau sans fil en envoyant un flux continu de données analogiques brutes, l'intelligence du système a été déportée au plus près du capteur afin de nous offrir plus de stabilité et d'efficacité. Le microcontrôleur ESP32 se charge désormais du pré-traitement numérique :

➔ **Filtrage par Hystérésis** : Pour contrer le bruit électronique et les micro-tremblements musculaires, un algorithme à double seuil a été implémenté. Un seuil haut déclenche l'action, et le système ne se réarme que lorsque le signal passe sous un seuil bas (repos total du muscle).

➔ **Machine à états (Logique Télérupteur)** : L'analyse physiologique a montré qu'il est difficile pour l'utilisateur de maintenir une tension musculaire maximale constante. L'ESP32 intègre donc une logique de "Toggle" : un pic de contraction modifie l'état du robot (ex: Fermeture de la pince), et le système maintient cet état même si le muscle se relâche, jusqu'au prochain pic de contraction (Ouverture).

Remarque: nous avons oublié de prendre une photo de la visualisation de cette algorithme à travers l'outil Serial Plot de Arduino, mais vous pouvez observer le rendu qu'on avait sur la figure 7 ci-dessus)

2-4-2. Communication TCP et Serveur ROS

Pour s'affranchir de l'ordinateur intermédiaire, nous avons utilisé le réseau Wi-Fi généré par le robot **Niryo** auquel **l'ESP32** se connecte en tant que client. Travailler sur un environnement ROS s'est alors avérée être le meilleur choix.

Ainsi, un script Python, agissant comme un **Nœud ROS** hybride et un **Serveur TCP**, tourne en permanence sur la **Raspberry Pi** du robot (écoute sur le port 5005).

Dès que **l'ESP32** valide une intention motrice, il envoie une **commande textuelle** minimaliste (par exemple "CLOSE" ou "FLEX"). Le serveur traduit instantanément cette chaîne de caractères en commandes articulaires (move_joints) ou matérielles (close_gripper) via l'API Niryo, garantissant une latence d'exécution quasi nulle.

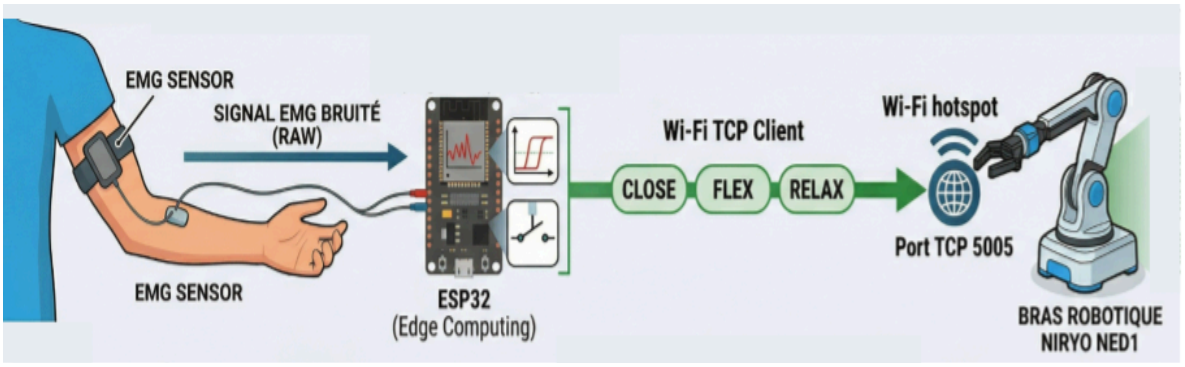


Figure 8 : Transfert de la commande textuelle via le réseau Wi-Fi

Le semestre dernier, cette migration vers ROS était justifiée par le besoin de contrôle en vitesse (cmd_vel) afin d'éviter les saccades de ***PyNiryo***. Néanmoins, nous n'avons pas pu mettre en place ce contrôle de vitesse mais ROS nous sert bien de **pont de communication** entre l'ESP32 et le robot, nous permettant ainsi de supprimer l'ordinateur de la boucle.

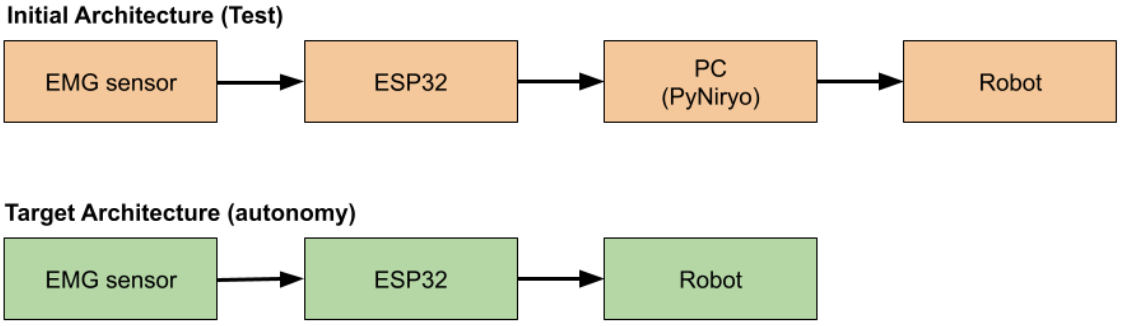


Figure 9 : Diagramme de comparaison des architectures (Target Architecture atteinte)

2-4-3. Modes de contrôles

Nous avions vu que pour répondre à différents cas d'usage, deux modes de contrôles ont été initialement prévus dans le contrôleur robotique :

- **Contrôle de l'articulation (Joint Control)** : ce mode nous permet un contrôle de bas niveau où notre signal EMG pilote directement la vitesse d'un seul moteur à la fois.

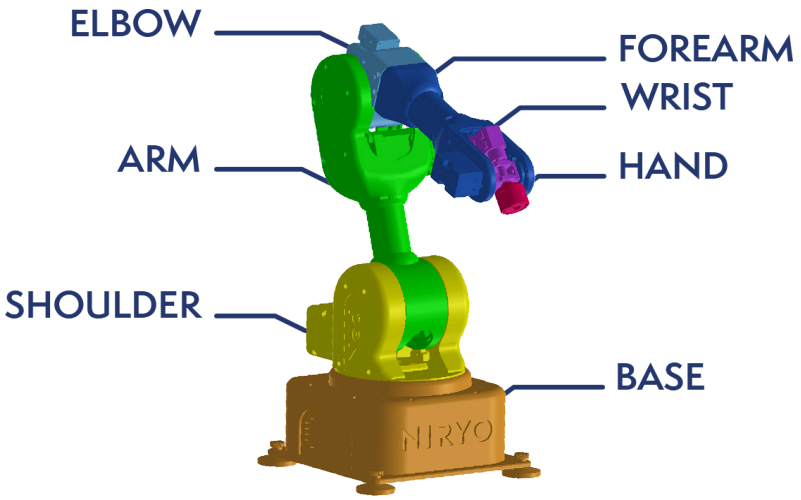


Figure 10 : Articulations du robot

- **Contrôle cartésien (Position Control / Cinématique Inverse)** : ce mode est plus intuitif pour l'utilisateur, car le signal EMG pilote le déplacement de la pince (l'effecteur final) selon les axes du monde (X, Y, Z). Le contrôleur interne du robot calcule alors en temps réel la position angulaire nécessaire pour chacun des 6 moteurs afin d'atteindre la coordonnée cible.

État d'avancement

Toutefois, par manque de temps et pour garantir la stabilité du système, nous sommes parvenus à réaliser uniquement le **contrôle de l'articulation** (le coude via le biceps) ainsi que l'ouverture/fermeture de la pince. Ce mode était plus idéal pour la démonstration technique et le débogage car il évite les complexités mathématiques liées à la cinématique inverse. Ainsi, la mise en œuvre complète du mode cartésien

et la mobilisation simultanée de plusieurs muscles restent des perspectives d'évolution pour ce projet.

2-4-4. Sécurité et gestion des erreurs

A travers ce projet, nous avons bien pu observer que l'utilisation de signaux physiologiques tels que l'EMG comporte intrinsèquement un risque de mouvements involontaires, principalement dus au **bruit électronique** ou à des **contractions musculaires parasites**. Par conséquent, intégrer des mécanismes de protection dans la chaîne de traitement était primordial pour garantir l'intégrité du matériel et la sécurité de l'utilisateur.

★ Filtrage par Hystérésis (Zone Morte)

Contrairement aux objectifs initiaux qui prévoyaient une « Deadzone » logicielle, la sécurité est désormais assurée de manière dynamique par l'algorithme **d'hystérésis** implémenté sur **l'ESP32**:

- **Élimination du bruit :** Le système utilise un double seuil (haut et bas) pour filtrer les micro-tremblements et les interférences électriques.
- **Seuil d'activation :** Le robot ne reçoit aucune commande tant que la contraction musculaire ne dépasse pas un seuil haut prédéfini, garantissant qu'aucun mouvement ne soit déclenché au repos.
- **Réarmement sécurisé :** Le système ne se réarme que lorsque le signal repasse sous un seuil bas (repos total), évitant ainsi les déclenchements saccadés liés à la fatigue musculaire.

★ Saturation de la vitesse et limites logicielles

Pour prévenir les réactions brusques du robot en cas de spasme musculaire ou de contraction soudaine, nous avons conservé et appliqué des limitations strictes :

- **Écrêtage des valeurs** : Les commandes de sortie sont saturées pour éviter des accélérations brutales.
- **Respect des limites physiques** : La vitesse maximale autorisée reste bridée en dessous des capacités physiques du robot afin de garantir des mouvements fluides et sécurisés.
- **Butées logicielles** : Chaque mouvement généré respecte strictement les limites articulaires (Joint Limits) définies par le constructeur pour éviter toute collision interne.

2-5. Résultats du projet à la fin du semestre 8

À l'issue de ce semestre, l'architecture globale (Acquisition → Traitement → Transmission → Exécution) est entièrement **fonctionnelle**. Nous avons réussi à démontrer le contrôle d'un bras robotique industriel à 6 axes via la contraction musculaire, sans aucun lien physique (câble) entre l'utilisateur et le robot.

Deux modes de contrôle ont été validés avec succès lors des démonstrations techniques :

- **Le contrôle de l'effecteur (Pince)** : Placé sur les muscles fléchisseurs de l'avant-bras, le système permet d'ouvrir et de fermer la pince du robot avec une excellente réactivité.
- **Le contrôle articulaire (Coude)** : Placé sur le biceps, le système contrôle dynamiquement la flexion et l'extension du bras robotique dans l'espace articulaire.

La modularité du code réseau (Sockets TCP) permet de basculer d'un mode à l'autre instantanément en ne modifiant que l'interface de commande côté serveur, validant ainsi la robustesse de notre architecture logicielle.

3. Conclusion

3-1. Bilan des problématiques rencontrées

Ce projet a démontré la faisabilité du contrôle d'un manipulateur robotique en utilisant des signaux électromyographiques (EMG). L'activité musculaire a été acquise, traitée et traduite avec succès en commandes robotiques simples grâce à une combinaison de capteurs EMG, d'un microcontrôleur embarqué et d'un framework de contrôle basé sur ROS. L'utilisation d'un microcontrôleur dédié a permis une numérisation et une prise de décision relativement fiable, palliant ainsi l'absence d'entrées analogiques sur le robot. Les résultats expérimentaux ont confirmé que les niveaux de contraction musculaire peuvent être détectés de manière robuste et associés à des actions robotiques.

3-2. Perspectives

Pour finaliser ce projet et reproduire parfaitement les mouvements d'un bras humain, il faudrait mettre en commun l'ensemble des travaux réalisés par notre groupe cette année. Les étapes d'acquisition de l'information et de communication ayant été testées avec succès, la prochaine étape consisterait à s'affranchir totalement de l'ordinateur. Cela nécessiterait la création d'une carte PCB dédiée à la gestion de la puissance ainsi qu'un boîtier imprimé en 3D, afin de stabiliser l'électronique (fixations instables qui nous créaient des faux contacts) tout en minimisant les parasites liés aux liaisons filaires.

Pour ce qui est de la partie informatique, l'implémentation de code pour réaliser des mouvements mobilisant plusieurs muscles est encore à réaliser (le code pour un unique capteur étant déjà fonctionnel) ainsi qu'un possible traitement numérique du signal issu des capteurs EMG réalisés, afin d'avoir un signal contenant au mieux l'information et obtenir ainsi une commande de mouvement plus stable et précise.

Enfin, pour pallier la variabilité extrême des signaux EMG, qu'elle soit liée à la fatigue musculaire, au placement des électrodes ou aux spécificités propres à chaque individu, un axe de développement majeur porterait sur l’automatisation de la calibration. En associant des algorithmes qui redéfineraient dynamiquement les seuils avec un apprentissage par intelligence artificielle, le système pourrait apprendre à reconnaître des signatures musculaires spécifiques en temps réel. Cette approche permettrait d'assurer une précision constante malgré l'amplitude variable des contractions rencontrées entre différentes personnes (inter-individus) mais également chez un même utilisateur.

4. Annexes

Sources :

- Site pour créer son propre capteur EMG consulté pour le notre: https://bluemoonshine.fun/Project-EMGSensor.php

- Video pour comprendre le redresseur de précision (point compliqué de la simulation):

  https://www.youtube.com/watch?v=5HweBajP-5g&t=61s

- Lien vers le *GitHub* du projet :

  https://github.com/LaureWOLFF/Projet_Bras

- Lien du site du capteur utilisé durant le projet:

  https://myoware.com/products/muscle-sensor/